Departamento de
Tecnología Digital

Tecsup TECNOLOGÍA CON SENTIDO



LABORATORIO N° 06

Desarrollo de aplicaciones Web: Implementación con AOP.

DOCENTE:

Coello Palomino, Ricardo

CURSO:

Desarrollo de aplicaciones Web
4 - C24 - Sección A - B

DESARROLLO DE APLICACIONES WEB AVANZADO: Implementación de Seguridad Básica.

I. Capacidades

Implementar AOP para auditoría y control de acceso en una API REST.

II. Seguridad

- En este laboratorio está prohibida la manipulación del hardware, conexiones eléctricas o de red. Así como la ingesta de alimentos y bebidas.
- Ubicar maletines y/o mochilas en lugar destinado para tal fin.
- Dejar la mesa de trabajo y la silla utilizada limpias y ordenadas.

III. Fundamento teórico

- Revise el material de la semana correspondiente antes del desarrollo del laboratorio.

IV. Normas empleadas

- *No aplica*

V. Recursos

- En este laboratorio, cada estudiante trabajará con una computadora con Windows 10.
- La instalación del software requerido se realizará en el equipo virtual.

VI. Metodología para el desarrollo de la tarea

- El desarrollo del laboratorio es individual.

VII. Procedimiento

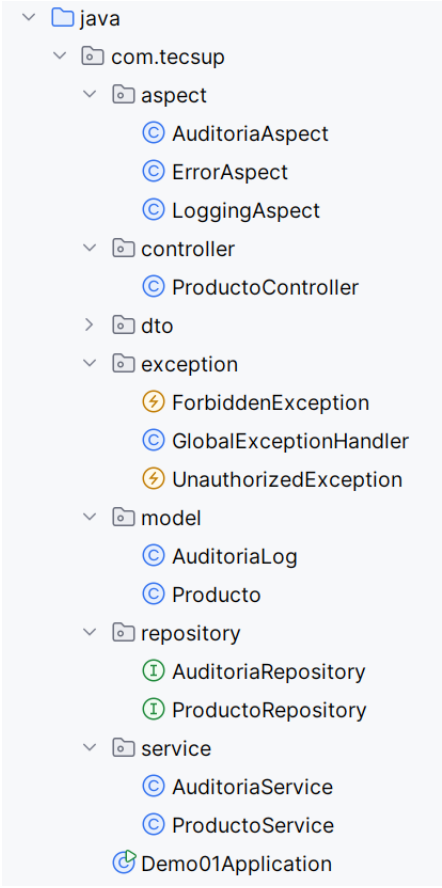
EXTENSIÓN DEL LABORATORIO: USUARIO Y ROL

1. Añadir el campo **usuario** a la tabla "**auditoria_log**".

```
ALTER TABLE auditoria_log ADD usuario VARCHAR(100);
```

2. Abrir el proyecto de la semana 5

3. Añadir las siguientes carpetas y archivos.



4. Modificar la clase **AuditoriaLog**

```
package com.tecsup.model;

import jakarta.persistence.*;
import java.time.LocalDateTime;

@Entity
public class AuditoriaLog {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String accion;
    private String metodo;
    private LocalDateTime fecha;
    private String detalle;
```

```
    private String usuario;

    public AuditoriaLog() {}

    public AuditoriaLog(String accion, String metodo, String detalle, String
usuario) {
        this.accion = accion;
        this.metodo = metodo;
        this.fecha = LocalDateTime.now();
        this.detalle = detalle;
        this.usuario = usuario;
    }

    public String getUsuario() {
        return usuario;
    }

    public void setUsuario(String usuario) {
        this.usuario = usuario;
    }

    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public String getAccion() {
        return accion;
    }

    public void setAccion(String accion) {
        this.accion = accion;
    }

    public String getMetodo() {
        return metodo;
    }

    public void setMetodo(String metodo) {
        this.metodo = metodo;
    }

    public LocalDateTime getFecha() {
        return fecha;
    }

    public void setFecha(LocalDateTime fecha) {
        this.fecha = fecha;
    }

    public String getDetalle() {
        return detalle;
    }

    public void setDetalle(String detalle) {
        this.detalle = detalle;
    }
}
```

5. En la clase **LoggingAspect** no hay cambios q realizar.

```
package com.tecsup.aspect;

import org.aspectj.lang.JoinPoint;
import org.aspectj.lang.annotation.*;
import org.springframework.stereotype.Component;

@Aspect
@Component
public class LoggingAspect {

    @Before("execution(* com.tecsup.service.*.*(..))")
    public void antes(JoinPoint joinPoint) {
        System.out.println("▶️ Ejecutando: " +
joinPoint.getSignature().getName());
    }

    @AfterReturning("execution(* com.tecsup.service.*.*(..))")
    public void despues(JoinPoint joinPoint) {
        System.out.println("✔️ Finalizó: " +
joinPoint.getSignature().getName());
    }
}
```

6. En la clase **ErrorAspect** no hay cambios que realizar.

```
package com.tecsup.aspect;

import org.aspectj.lang.annotation.*;
import org.springframework.stereotype.Component;

@Aspect
@Component
public class ErrorAspect {

    @AfterThrowing(
            pointcut = "execution(* com.tecsup.service.*.*(..))",
            throwing = "ex"
    )
    public void capturarError(Exception ex) {
        System.out.println("❌ ERROR AOP: " + ex.getMessage());
    }
}
```

7. En la interface **AuditoriaRepository** no hay cambios q realizar.

```
package com.tecsup.repository;

import com.tecsup.model.AuditoriaLog;
import org.springframework.data.jpa.repository.JpaRepository;

public interface AuditoriaRepository extends JpaRepository<AuditoriaLog, Long> {
}
```

8. Modificar la clase **AuditoriaService**

```
package com.tecsup.service;

import com.tecsup.model.AuditoriaLog;
import com.tecsup.repository.AuditoriaRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

@Service
public class AuditoriaService {

    @Autowired
    private AuditoriaRepository repo;

    public void registrar(String accion, String metodo, String detalle,String
usuario) {
        AuditoriaLog log = new AuditoriaLog(accion, metodo, detalle, usuario);
        repo.save(log);
    }
}
```

9. Crear la clase **ForbiddenException**

```
package com.tecsup.exception;

public class ForbiddenException extends RuntimeException {
    public ForbiddenException(String mensaje) {
        super(mensaje);
    }
}
```

10. Crear la clase **UnauthorizedException**

```
package com.tecsup.exception;
public class UnauthorizedException extends RuntimeException {
    public UnauthorizedException(String mensaje) {
        super(mensaje);
    }
}
```

11. Modificar la clase **GlobalExceptionHandler**

```
package com.tecsup.exception;

import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.*;

import java.util.HashMap;
import java.util.Map;

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(MethodArgumentNotValidException.class)
```

```
    public ResponseEntity<Map<String, String>>
manejarErrores(MethodArgumentNotValidException ex) {

        Map<String, String> errores = new HashMap<>();

        ex.getBindingResult().getFieldErrors().forEach(error -> {
            errores.put(error.getField(), error.getDefaultMessage());
        });

        return ResponseEntity.badRequest().body(errores);
    }


@ExceptionHandler(UnauthorizedException.class)
public ResponseEntity<String> manejarUnauthorized(UnauthorizedException ex) {
    return ResponseEntity.status(401).body(ex.getMessage());
}

    @ExceptionHandler(ForbiddenException.class)
    public ResponseEntity<String> manejarForbidden(ForbiddenException ex) {
        return ResponseEntity.status(403).body(ex.getMessage());
    }
}
```

12. Modificar la clase **AuditoriaAspect**

```
package com.tecsup.aspect;

import com.tecsup.exception.ForbiddenException;
import com.tecsup.exception.UnauthorizedException;
import com.tecsup.model.Producto;
import com.tecsup.service.AuditoriaService;
import jakarta.servlet.http.HttpServletRequest;
import org.aspectj.lang.JoinPoint;
import org.aspectj.lang.annotation.*;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Component;

import java.util.Map;

@Aspect
@Component
public class AuditoriaAspect {

   @Autowired
   private AuditoriaService auditoriaService;

   @Autowired
   private HttpServletRequest request;

   // Usuarios centralizados
   private Map<String, String> usuarios = Map.of(
        "Ricardo", "ADMIN",
        "Ana", "USER",
        "Luis", "USER"
   );

   // ========================
   // VALIDACIÓN
```

```
// ========================

private void validarUsuario() {

    String usuario = request.getHeader("Usuario");
    String rolHeader = request.getHeader("Rol");

    if (usuario == null || rolHeader == null) {
        throw new UnauthorizedException("Debe enviar Usuario y Rol");
    }

    String rolReal = usuarios.get(usuario);

    if (rolReal == null || !rolReal.equals(rolHeader)) {
        throw new ForbiddenException("No tiene permisos");
    }
}

private void validarRol(String... rolesPermitidos) {

    validarUsuario();

    String rol = request.getHeader("Rol");

    for (String permitido : rolesPermitidos) {
        if (permitido.equals(rol)) {
            return;
        }
    }

    throw new ForbiddenException("Acceso denegado");
}

// ========================
// USUARIO
// ========================

private String obtenerUsuario() {

    String usuario = request.getHeader("Usuario");

    if (usuario == null) return "ANONIMO";

    String rol = usuarios.get(usuario);

    return (rol != null)
            ? usuario + " (" + rol + ")"
            : "DESCONOCIDO";
}

// ========================
// CONTROL POR MÉTODO
// ========================

@Before("execution(* com.tecsup.service.ProductoService.guardar(..))")
public void validarCrear() {
    validarRol("ADMIN");
```

```
}

@Before("execution(* com.tecsup.service.ProductoService.eliminar(..))")
public void validarEliminar() {
   validarRol("ADMIN");
}

@Before("execution(* com.tecsup.service.ProductoService.actualizar(..))")
public void validarActualizar() {
   validarRol("ADMIN");
}

@Before("execution(* com.tecsup.service.ProductoService.listar(..))")
public void validarListar() {
   validarRol("ADMIN", "USER");
}

// ========================
// AUDITORÍA
// ========================

@AfterReturning("execution(* com.tecsup.service.ProductoService.guardar(..))")
public void auditarGuardar(JoinPoint joinPoint) {

   auditoriaService.registrar(
         "CREAR",
         joinPoint.getSignature().getName(),
         "Se creó producto: " + obtenerParametros(joinPoint),
         obtenerUsuario()
   );
}

@AfterReturning("execution(* com.tecsup.service.ProductoService.eliminar(..))")
public void auditarEliminar(JoinPoint joinPoint) {

   auditoriaService.registrar(
         "ELIMINAR",
         joinPoint.getSignature().getName(),
         "Se eliminó producto ID: " + obtenerParametros(joinPoint),
         obtenerUsuario()
   );
}

@AfterReturning("execution(* com.tecsup.service.ProductoService.actualizar(..))")
public void auditarActualizar(JoinPoint joinPoint) {

   auditoriaService.registrar(
         "ACTUALIZAR",
         joinPoint.getSignature().getName(),
         "Se actualizó producto: " + obtenerParametros(joinPoint),
         obtenerUsuario()
   );
}

// ========================
// UTILIDAD (VERSIÓN CORRECTA)
// ========================
```

```
    private String obtenerParametros(JoinPoint joinPoint) {
        Object[] args = joinPoint.getArgs();

        if (args.length == 0) return "sin datos";

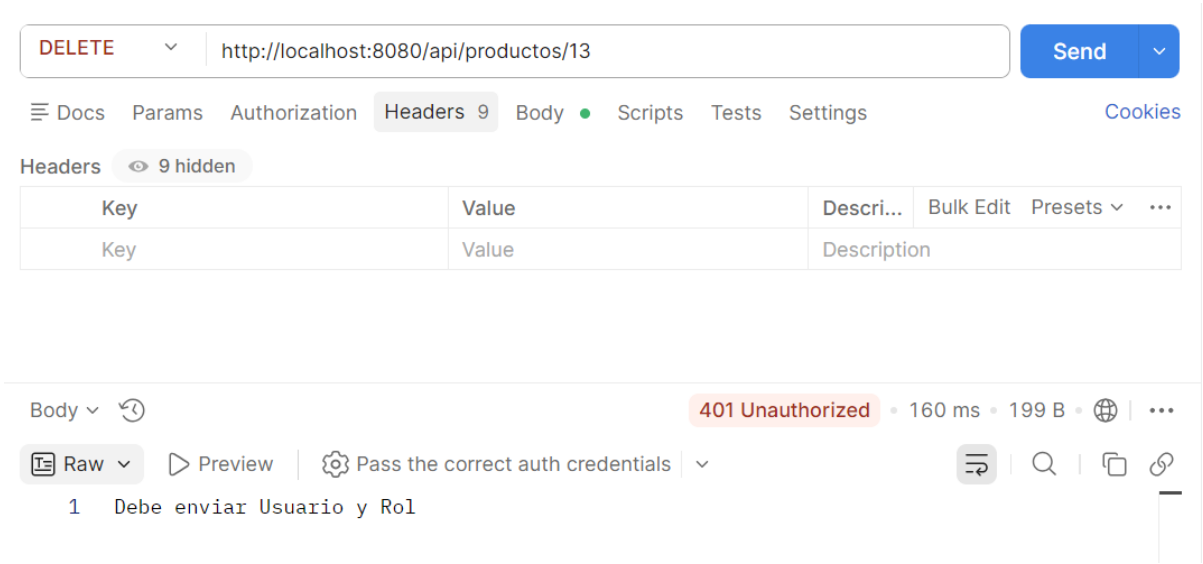
        Object obj = args[0];

        if (obj instanceof Producto p) {
            return "nombre=" + p.getNombre() +
                    ", precio=" + p.getPrecio();
        }

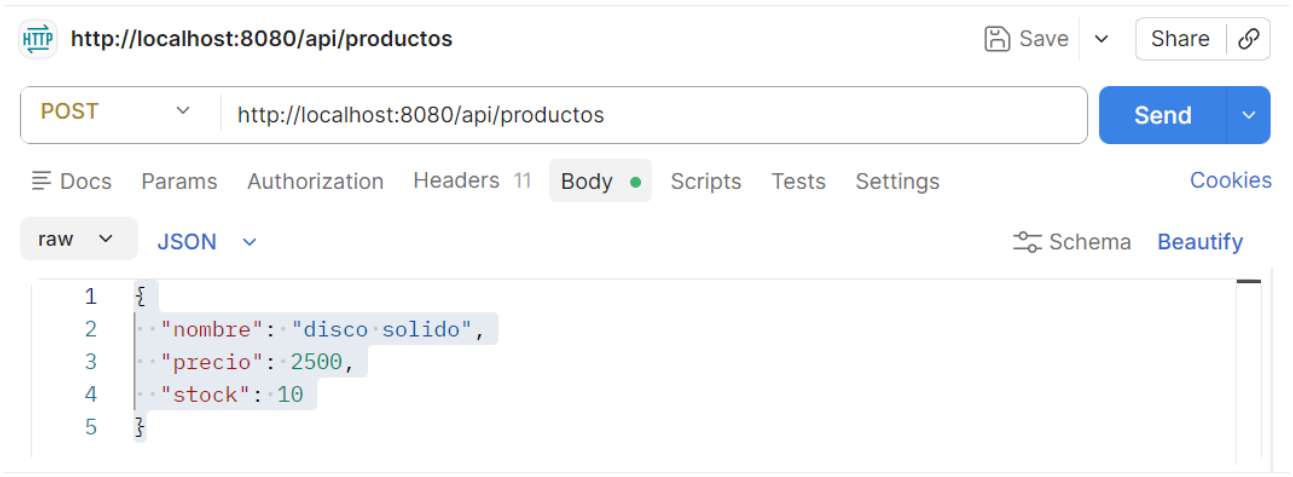
        return obj.toString();
    }
}
```

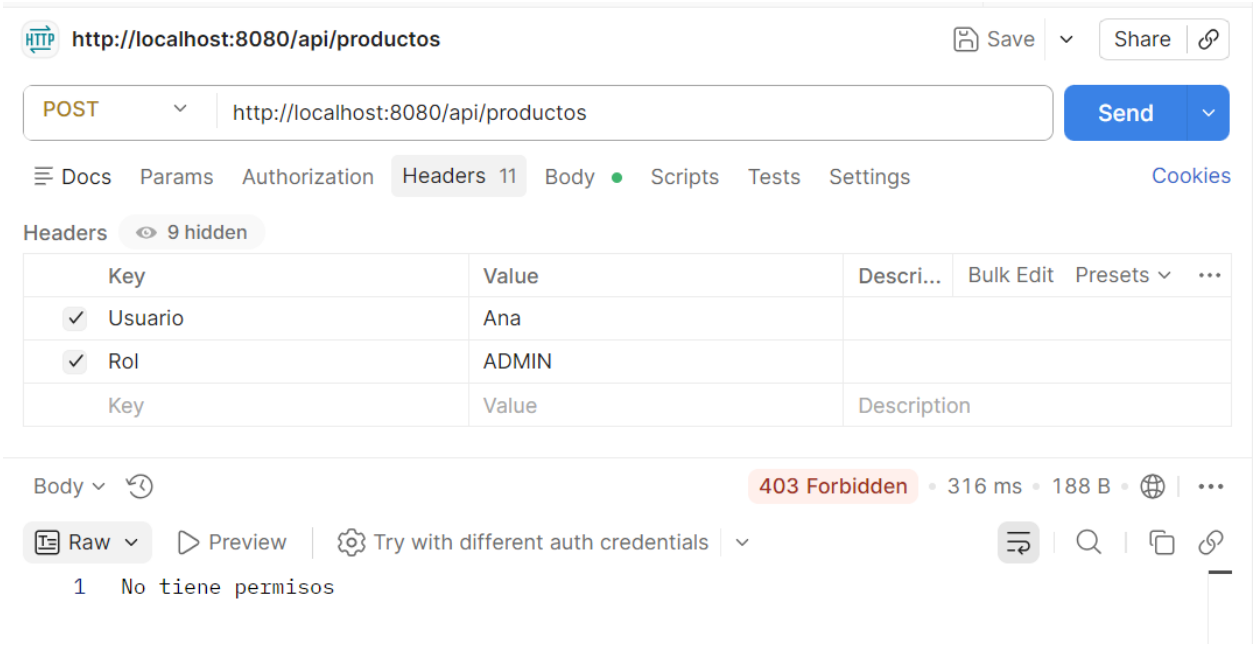
8. probar en **POSTMAN**

DELETE

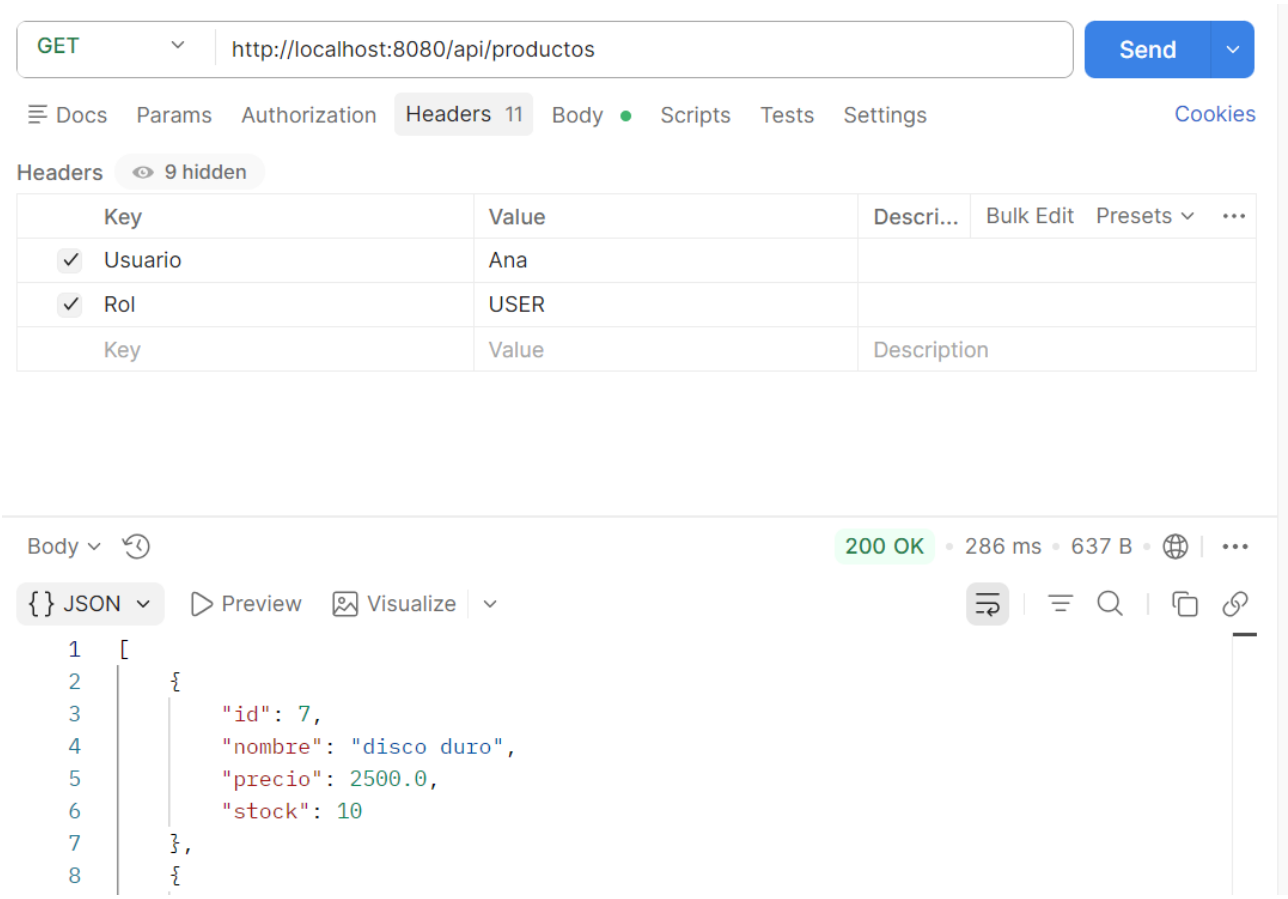


POST





GET



El rol ADMIN realiza todo el CRUD.

El rol USER realizo solo el listado.

EVIDENCIA

MySQL ha devuelto un conjunto de valores vacío (es decir: cero columnas). (La consulta tardó 0,0047 segundos.)

```
ALTER TABLE auditoria_log ADD usuario VARCHAR(100);
```

[Editar en línea] [Editar] [Crear código PHP]

ErrorAspect.java | Demo01Application.java | AuditoriaLog.java

src > main > java > com > tecsup > model > AuditoriaLog.java > AuditoriaLog

```java
package com.tecsup.model;

import jakarta.persistence.*;
import java.time.LocalDateTime;

@Entity
public class AuditoriaLog {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String accion;
    private String metodo;
    private LocalDateTime fecha;
    private String detalle;
    private String usuario; // Nuevo campo agregado

    public AuditoriaLog() {}

    public AuditoriaLog(String accion, String metodo, String detalle, String usuario) {
        this.accion = accion;
        this.metodo = metodo;
        this.fecha = LocalDateTime.now();
        this.detalle = detalle;
        this.usuario = usuario;
    }

    public String getUsuario() {
        return usuario;
    }

    public void setUsuario(String usuario) {
```

ErrorAspect.java | Demo01Application.java | AuditoriaLog.java | ForbiddenException.java U

src > main > java > com > tecsup > exception > ForbiddenException.java > ForbiddenException

```java
package com.tecsup.exception;

public class ForbiddenException extends RuntimeException {
    public ForbiddenException(String mensaje) {
        super(mensaje);
    }
}
```

ErrorAspect.java | Demo01Application.java | AuditoriaLog.java | ForbiddenException.java U

src > main > java > com > tecsup > exception > UnauthorizedException.java > UnauthorizedException

```java
package com.tecsup.exception;

public class UnauthorizedException extends RuntimeException {
    public UnauthorizedException(String mensaje) {
        super(mensaje);
    }
}
```

Demo01Application.java | AuditoriaLog.java | ForbiddenException.java U | UnauthorizedException.java U | GlobalExceptionHandler.java

src > main > java > com > tecsup > exception > GlobalExceptionHandler.java > Language Support for Java(TM) by Red Hat > GlobalExceptionHandler

```java
package com.tecsup.exception;

import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.*;
import java.util.HashMap;
import java.util.Map;

@RestControllerAdvice
public class GlobalExceptionHandler {

    // Manejador de errores de validación (El que ya teníamos)
    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<Map<String, String>> manejarErrores(MethodArgumentNotValidException ex) {
        Map<String, String> errores = new HashMap<>();
        ex.getBindingResult().getFieldErrors().forEach(error -> {
            errores.put(error.getField(), error.getDefaultMessage());
        });
        return ResponseEntity.badRequest().body(errores);
    }

    // NUEVO: Manejador para usuarios no autorizados (Faltan credenciales)
    @ExceptionHandler(UnauthorizedException.class)
    public ResponseEntity<String> manejarUnauthorized(UnauthorizedException ex) {
        return ResponseEntity.status(401).body(ex.getMessage());
    }

    // NUEVO: Manejador para usuarios sin permisos (Falta de Rol)
    @ExceptionHandler(ForbiddenException.class)
    public ResponseEntity<String> manejarForbidden(ForbiddenException ex) {
        return ResponseEntity.status(403).body(ex.getMessage());
```

Actividad

- Permitir que USER también pueda crear productos
- Registrar auditoría en el método listar()
  Debe guardar:
  Acción: LISTAR
  Cantidad de productos
- Mejorar auditoría en `eliminar()`
  Mostrar:
  `Se eliminó producto ID: X`

```
// --- ACTIVIDAD: Permitir que USER también pueda crear productos ---
@Before("execution(* com.tecsup.service.ProductoService.guardar(..))")
public void verificarPermisosCrear(JoinPoint joinPoint) {
    String rol = request.getHeader("Rol");

    if (rol == null || rol.isEmpty()) {
        throw new UnauthorizedException(mensaje: "Falta indicar el Rol en los Headers");
    }

    // Aceptamos tanto ADMIN como USER
    if (!rol.equalsIgnoreCase(anotherString: "ADMIN") && !rol.equalsIgnoreCase(anotherString: "USER")) {
        throw new ForbiddenException("El rol " + rol + " no tiene permisos para crear productos");
    }
}

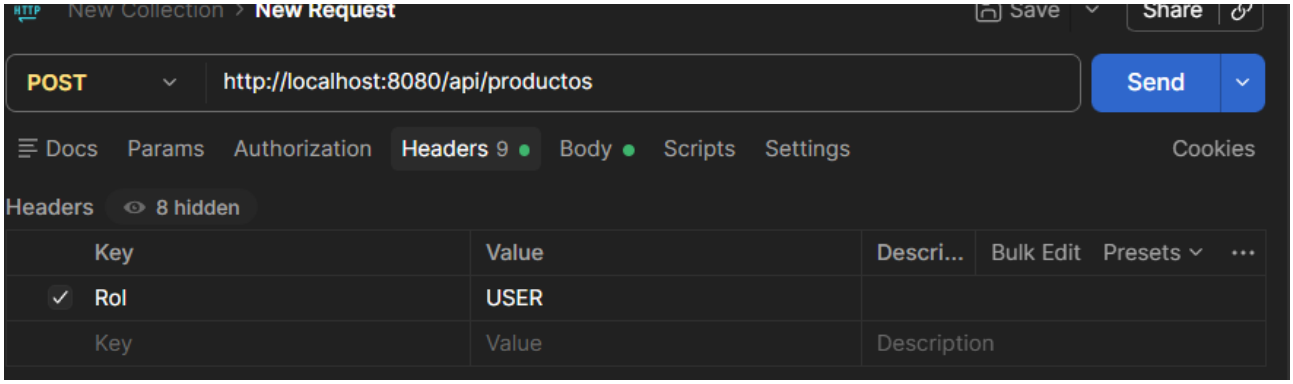
@AfterReturning("execution(* com.tecsup.service.ProductoService.guardar(..))")
public void auditarGuardar(JoinPoint joinPoint) {
    auditoriaService.registrar(accion: "CREAR", joinPoint.getSignature().getName(), detalle: "Se registró un producto");
}
```

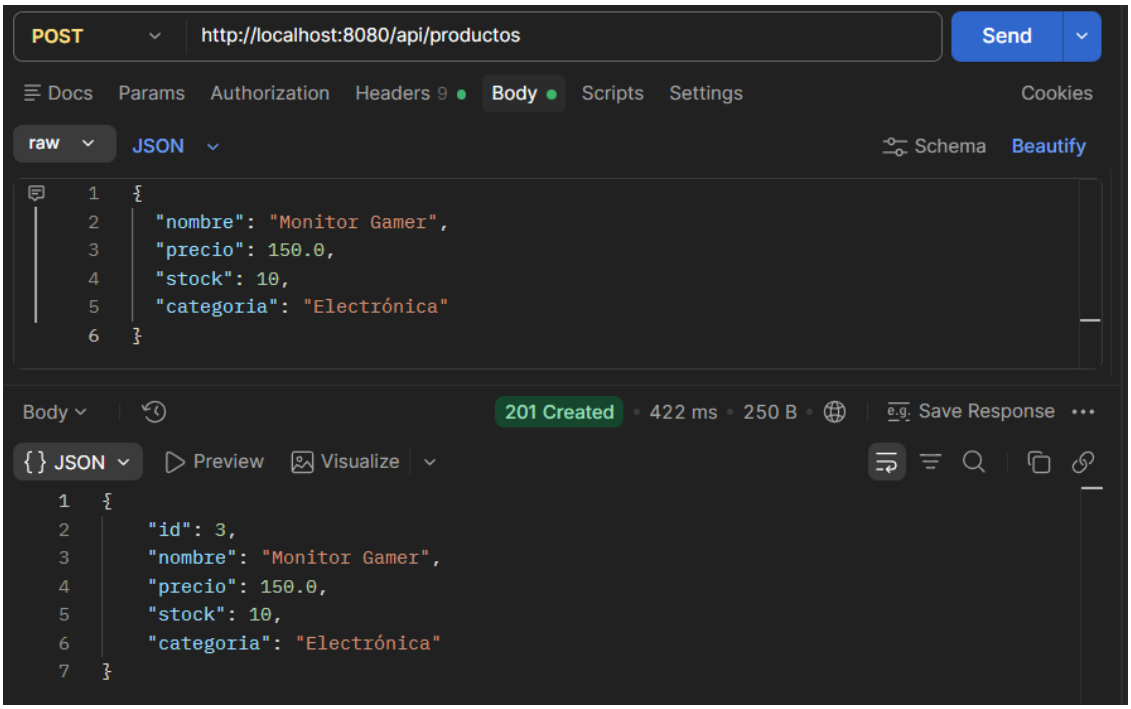
```
// --- ACTIVIDAD: Mejorar auditoría en eliminar() ---
@AfterReturning("execution(* com.tecsup.service.ProductoService.eliminar(..))")
public void auditarEliminar(JoinPoint joinPoint) {
    Object[] args = joinPoint.getArgs();
    Long id = (args.length > 0) ? (Long) args[0] : null;

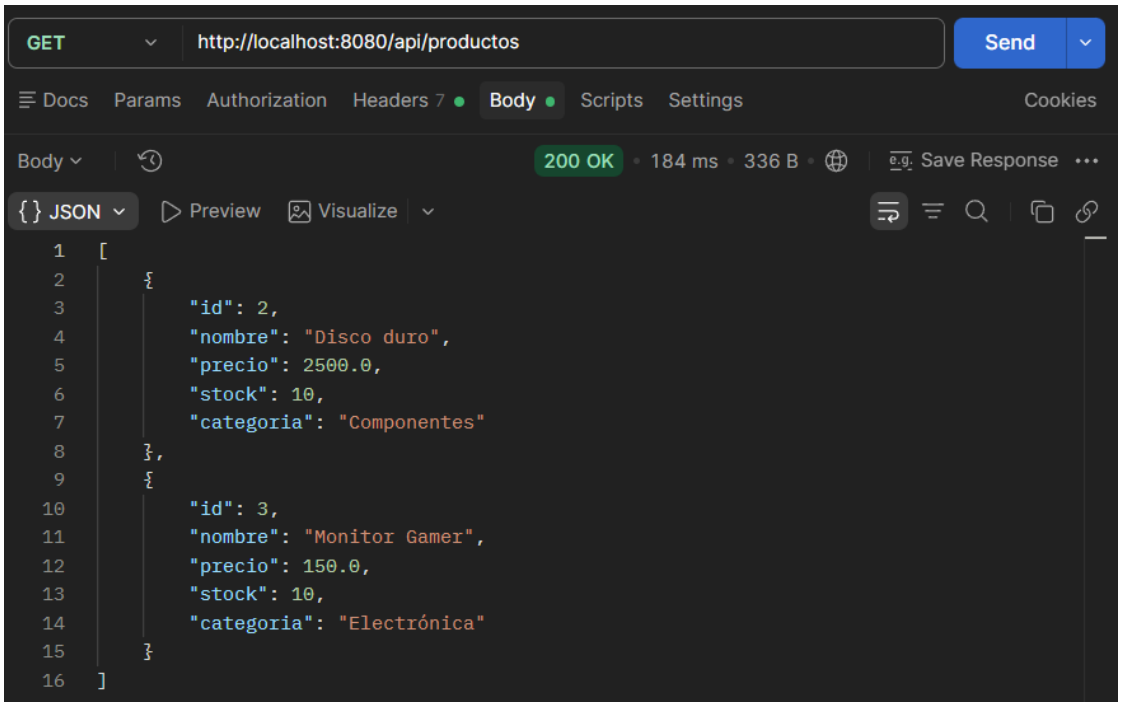
    auditoriaService.registrar(
            accion: "ELIMINAR",
            joinPoint.getSignature().getName(),
            "Se eliminó producto ID: " + id // Texto exacto según tu imagen
    );
}
```

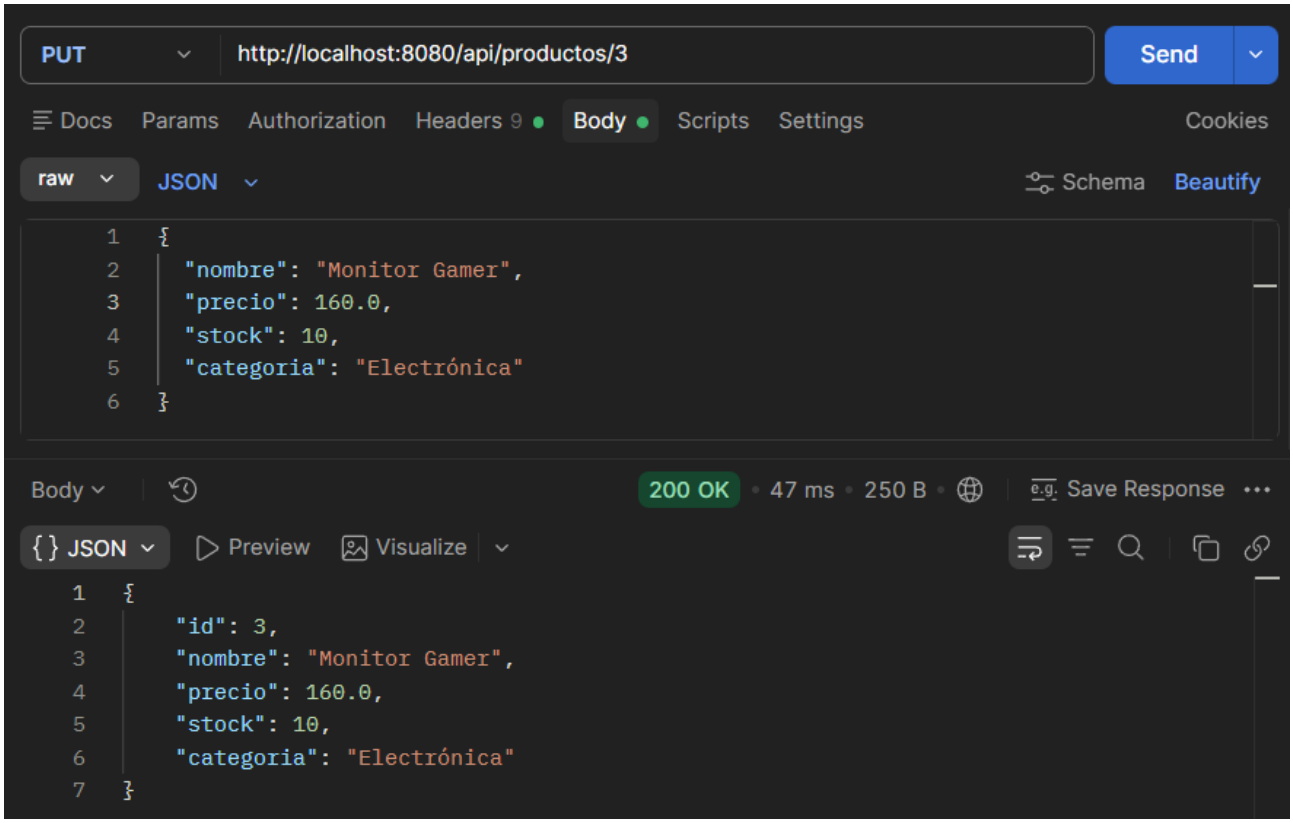
```
// --- ACTIVIDAD: Registrar auditoría en listar() ---
@AfterReturning(pointcut = "execution(* com.tecsup.service.ProductoService.listar(..))", returning = "lista")
public void auditarListar(JoinPoint joinPoint, Object lista) {
    int cantidad = 0;
    if (lista instanceof List) {
        cantidad = ((List<?>) lista).size();
    }

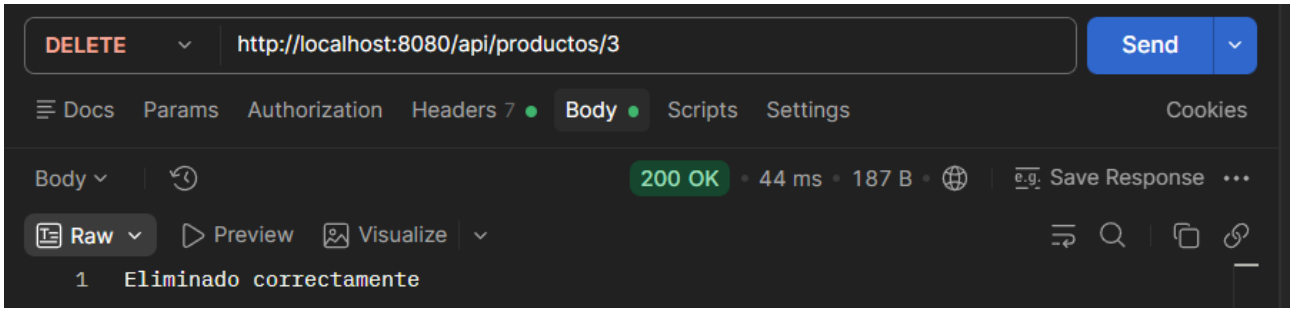
    auditoriaService.registrar(
            accion: "LISTAR",
            joinPoint.getSignature().getName(),
            "Cantidad de productos: " + cantidad // Texto exacto según tu imagen
    );
```


New Collection > New Request
Save
Share
POST
http://localhost:8080/api/productos
Send
Docs
Params
Authorization
Headers 9
Body
Scripts
Settings
Cookies
Headers
8 hidden
Key
Value
Descri...
Bulk Edit
Presets
Rol
USER
Key
Value
Description


POST
http://localhost:8080/api/productos
Send
Docs
Params
Authorization
Headers 9
Body
Scripts
Settings
Cookies
raw
JSON
Schema
Beautify
{
"nombre": "Monitor Gamer",
"precio": 150.0,
"stock": 10,
"categoria": "Electrónica"
}
Body
201 Created
422 ms
250 B
Save Response
{} JSON
Preview
Visualize
{
"id": 3,
"nombre": "Monitor Gamer",
"precio": 150.0,
"stock": 10,
"categoria": "Electrónica"
}


GET
http://localhost:8080/api/productos
Send
Docs
Params
Authorization
Headers 7
Body
Scripts
Settings
Cookies
Body
200 OK
184 ms
336 B
Save Response
{} JSON
Preview
Visualize
[
{
"id": 2,
"nombre": "Disco duro",
"precio": 2500.0,
"stock": 10,
"categoria": "Componentes"
},
{
"id": 3,
"nombre": "Monitor Gamer",
"precio": 150.0,
"stock": 10,
"categoria": "Electrónica"
}
]







	id	accion	metodo	fecha	detalle	usuario
Editar Copiar Borrar	1	CREAR	guardar	2026-09-28 02:44:32	Se registró un producto	NULL
Editar Copiar Borrar	2	CREAR	guardar	2026-09-28 02:45:27	Se registró un producto	NULL
Editar Copiar Borrar	3	ACTUALIZAR	actualizar	2026-09-28 02:45:27	Se actualizó producto con ID: 1	NULL
Editar Copiar Borrar	4	LISTAR	listar	2026-09-28 02:46:08	Se obtuvieron 2 registros	NULL
Editar Copiar Borrar	5	ELIMINAR	eliminar	2026-09-28 02:46:39	Se eliminó producto con ID: 1	NULL
Editar Copiar Borrar	6	CREAR	guardar	2026-09-28 11:31:20	Se registró un producto	NULL
Editar Copiar Borrar	7	LISTAR	listar	2026-09-28 11:33:10	Cantidad de productos: 2	NULL
Editar Copiar Borrar	8	CREAR	guardar	2026-09-28 11:36:04	Se registró un producto	NULL
Editar Copiar Borrar	9	ACTUALIZAR	actualizar	2026-09-28 11:36:04	Se actualizó producto con ID: 3	NULL
Editar Copiar Borrar	10	ELIMINAR	eliminar	2026-09-28 11:36:28	Se eliminó producto ID: 3	NULL

OBSERVACIONES: *(Las observaciones son las notas aclaratorias y problemas que se pudo presentar en el desarrollo del laboratorio)*

Mencionar 5 observaciones:

- **Al principio me confundí un poco al probar la seguridad en Postman, pero entendí que el "Rol" debía enviarse obligatoriamente desde la pestaña *Headers* y no en el *Body* para que el sistema me dejara crear el producto.**
- **Noté que inyectar HttpServletRequest directamente en la clase del Aspecto fue una solución muy ingeniosa, ya que me permitió leer quién hacía la petición sin tener que tocar ni ensuciar el código de mis controladores.**
- **Me sorprendió lo fácil que fue implementar el buscador; con solo escribir findByNombreContainingIgnoreCase en la interfaz del repositorio, Spring construyó toda la consulta SQL por detrás sin que yo escribiera nada más.**
- **Pude comprobar que mis validaciones anteriores siguen intactas. Cuando intenté mandar un producto dejando la categoría vacía, el @NotBlank del DTO saltó de inmediato y me devolvió el error 400.**
- **Me di cuenta de que usar joinPoint.getArgs() es la técnica clave para poder atrapar datos dinámicos (como el ID del producto que se está eliminando o la cantidad de una lista) justo en el momento en que ocurre la acción.**

CONCLUSIONES: *(Las conclusiones son una opinión sobre tu trabajo, explicar cómo resolviste las dudas o problemas presentados en el laboratorio. Además de aportar una opinión critica de lo realizado)*

Indicar 5 conclusiones:

- **Entendí que la Programación Orientada a Aspectos (AOP) es súper poderosa porque me permitió meter un sistema de auditoría y validación de roles de forma "invisible", manteniendo el código de mi API totalmente limpio.**
- **Hacer la validación de usuarios leyendo las cabeceras HTTP me dio una idea muy clara de cómo funcionan los sistemas de seguridad y permisos reales en el backend.**
- **Concluyo que aprovechar los *Query Methods* automáticos de Spring Data JPA para hacer búsquedas agiliza un montón el desarrollo y evita que cometamos errores de tipeo armando sentencias SQL a mano.**
- **Me queda clarísimo que usar DTOs sigue siendo indispensable; agregar el campo categoría me demostró lo fácil que es moldear y validar exactamente lo que quiero recibir sin exponer mi base de datos.**
- **Al juntar todo lo de esta sesión, comprobé que probar las rutas en Postman (POST con headers, GET para buscar, etc.) es el paso definitivo para garantizar que todas las capas (Controller, Aspect, Service y MySQL) están perfectamente conectadas y funcionando.**

Departamento de
Tecnología Digital

Tecsup TECNOLOGÍA CON SENTIDO



LABORATORIO N° 06

Desarrollo de aplicaciones Web: Implementación con AOP.

DOCENTE:

Coello Palomino, Ricardo

CURSO:

Desarrollo de aplicaciones Web
4 - C24 - Sección A - B

DESARROLLO DE APLICACIONES WEB AVANZADO: Implementación con AOP.

I. Capacidades

Implementar AOP para auditoría y control de acceso en una API REST.

II. Seguridad

- En este laboratorio está prohibida la manipulación del hardware, conexiones eléctricas o de red. Así como la ingesta de alimentos y bebidas.
- Ubicar maletines y/o mochilas en lugar destinado para tal fin.
- Dejar la mesa de trabajo y la silla utilizada limpias y ordenadas.

III. Fundamento teórico

- Revise el material de la semana correspondiente antes del desarrollo del laboratorio.

IV. Normas empleadas

- *No aplica*

V. Recursos

- En este laboratorio, cada estudiante trabajará con una computadora con Windows 10.
- La instalación del software requerido se realizará en el equipo virtual.

VI. Metodología para el desarrollo de la tarea

- El desarrollo del laboratorio es individual.

VII. Procedimiento

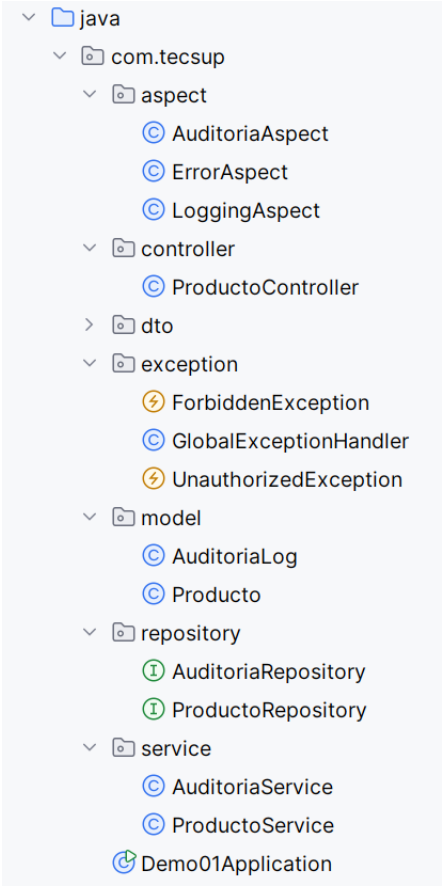
EXTENSIÓN DEL LABORATORIO: AOP SOBRE API DE PRODUCTOS

1. Añadir la siguiente tabla en la base de datos.

```
CREATE TABLE auditoria_log (
id BIGINT AUTO_INCREMENT PRIMARY KEY,
accion VARCHAR(50),
metodo VARCHAR(100),
fecha TIMESTAMP,
detalle VARCHAR(255)
);
```

2. Abrir el proyecto de la semana 5

3. Añadir las siguientes carpetas y archivos.



2. Crear la clase **AuditoriaLog**

```java
package com.tecsup.model;

import jakarta.persistence.*;
import java.time.LocalDateTime;

@Entity
public class AuditoriaLog {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
```

```
    private String accion;
    private String metodo;
    private LocalDateTime fecha;
    private String detalle;

    public AuditoriaLog() {}

    public AuditoriaLog(String accion, String metodo, String detalle) {
        this.accion = accion;
        this.metodo = metodo;
        this.fecha = LocalDateTime.now();
        this.detalle = detalle;
    }


    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public String getAccion() {
        return accion;
    }

    public void setAccion(String accion) {
        this.accion = accion;
    }

    public String getMetodo() {
        return metodo;
    }

    public void setMetodo(String metodo) {
        this.metodo = metodo;
    }

    public LocalDateTime getFecha() {
        return fecha;
    }

    public void setFecha(LocalDateTime fecha) {
        this.fecha = fecha;
    }

    public String getDetalle() {
        return detalle;
    }

    public void setDetalle(String detalle) {
        this.detalle = detalle;
    }
}
```

3. Crear la clase LoggingAspect

```
package com.tecsup.aspect;

import org.aspectj.lang.JoinPoint;
import org.aspectj.lang.annotation.*;
import org.springframework.stereotype.Component;

@Aspect
@Component
public class LoggingAspect {

    @Before("execution(* com.tecsup.service.*.*(..))")
    public void antes(JoinPoint joinPoint) {
        System.out.println("▶ Ejecutando: " +
joinPoint.getSignature().getName());
    }

    @AfterReturning("execution(* com.tecsup.service.*.*(..))")
    public void despues(JoinPoint joinPoint) {
        System.out.println("✔ Finalizó: " +
joinPoint.getSignature().getName());
    }
}
```

4. Crear la clase ErrorAspect

```
package com.tecsup.aspect;

import org.aspectj.lang.annotation.*;
import org.springframework.stereotype.Component;

@Aspect
@Component
public class ErrorAspect {

    @AfterThrowing(
            pointcut = "execution(* com.tecsup.service.*.*(..))",
            throwing = "ex"
    )
    public void capturarError(Exception ex) {
        System.out.println("ERROR AOP: " + ex.getMessage());
    }
}
```

5. Modificar la clase **GlobalExceptionHandler**

```
package com.tecsup.exception;

import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.*;

import java.util.HashMap;
import java.util.Map;
```

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<Map<String, String>>
manejarErrores(MethodArgumentNotValidException ex) {

        Map<String, String> errores = new HashMap<>();

        ex.getBindingResult().getFieldErrors().forEach(error -> {
            errores.put(error.getField(), error.getDefaultMessage());
        });

        return ResponseEntity.badRequest().body(errores);
    }
}
```

6. Crear la interface **AuditoriaRepository**

```
package com.tecsup.repository;

import com.tecsup.model.AuditoriaLog;
import org.springframework.data.jpa.repository.JpaRepository;

public interface AuditoriaRepository extends JpaRepository<AuditoriaLog, Long> {
}
```

7. Crear la clase **AuditoriaService**

```
package com.tecsup.service;

import com.tecsup.model.AuditoriaLog;
import com.tecsup.repository.AuditoriaRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

@Service
public class AuditoriaService {

    @Autowired
    private AuditoriaRepository repo;

    public void registrar(String accion, String metodo, String detalle) {
        AuditoriaLog log = new AuditoriaLog(accion, metodo, detalle);
        repo.save(log);
    }
}
```

8. Crear la clase **AuditoriaAspect**

```
package com.tecsup.aspect;

import com.tecsup.service.AuditoriaService;
import org.aspectj.lang.JoinPoint;
import org.aspectj.lang.annotation.*;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Component;
```

```
@Aspect
@Component
public class AuditoriaAspect {

    @Autowired
    private AuditoriaService auditoriaService;

    @AfterReturning("execution(*
com.tecsup.service.ProductoService.guardar(..))")
    public void auditarGuardar(JoinPoint joinPoint) {

        auditoriaService.registrar(
                "CREAR",
                joinPoint.getSignature().getName(),
                "Se registró un producto"
        );
    }

    @AfterReturning("execution(*
com.tecsup.service.ProductoService.eliminar(..))")
    public void auditarEliminar(JoinPoint joinPoint) {

        auditoriaService.registrar(
                "ELIMINAR",
                joinPoint.getSignature().getName(),
                "Se eliminó un producto"
        );
    }
}
```

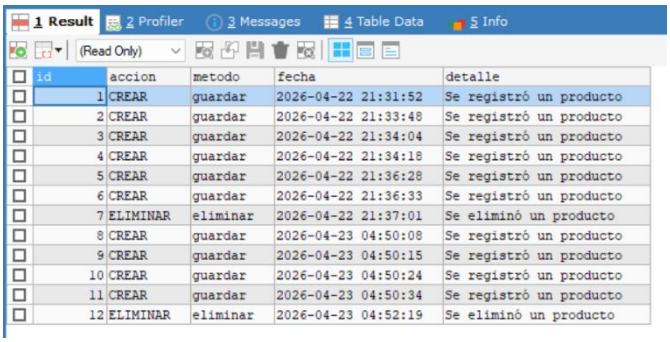
9. probar en **POSTMAN**

Post: http://localhost:8080/api/productos

```
{
  "nombre": "disco duro",
  "precio": 2500,
  "stock": 10
}
```

Delete: http://localhost:8080/api/productos/1

10. Revisar base de datos:



1 Result | 2 Profiler | 3 Messages | 4 Table Data | 5 Info

(Read Only)

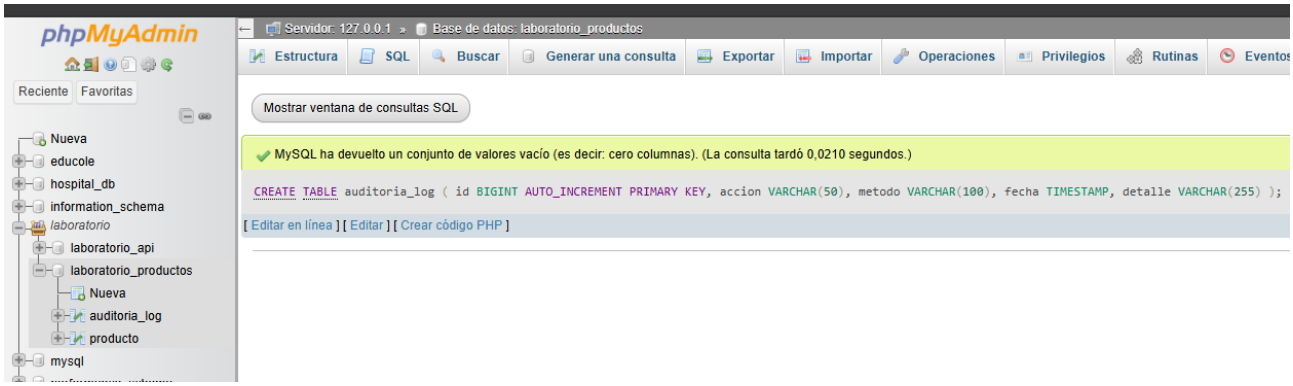
id	accion	metodo	fecha	detalle
1	CREAR	guardar	2026-04-22 21:31:52	Se registró un producto
2	CREAR	guardar	2026-04-22 21:33:48	Se registró un producto
3	CREAR	guardar	2026-04-22 21:34:04	Se registró un producto
4	CREAR	guardar	2026-04-22 21:34:18	Se registró un producto
5	CREAR	guardar	2026-04-22 21:36:28	Se registró un producto
6	CREAR	guardar	2026-04-22 21:36:33	Se registró un producto
7	ELIMINAR	eliminar	2026-04-22 21:37:01	Se eliminó un producto
8	CREAR	guardar	2026-04-23 04:50:08	Se registró un producto
9	CREAR	guardar	2026-04-23 04:50:15	Se registró un producto
10	CREAR	guardar	2026-04-23 04:50:24	Se registró un producto
11	CREAR	guardar	2026-04-23 04:50:34	Se registró un producto
12	ELIMINAR	eliminar	2026-04-23 04:52:19	Se eliminó un producto

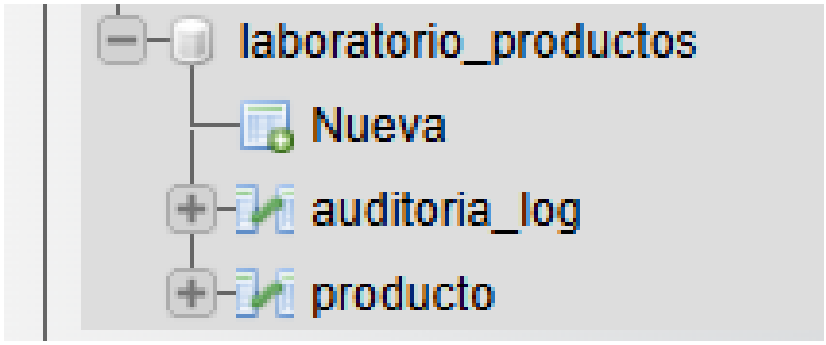
ACTIVIDAD DE APRENDIZAJE

INSTRUCCIONES

Extender la clase AuditoriaAspect para registrar también:

- Método actualizar()
- Método listar()





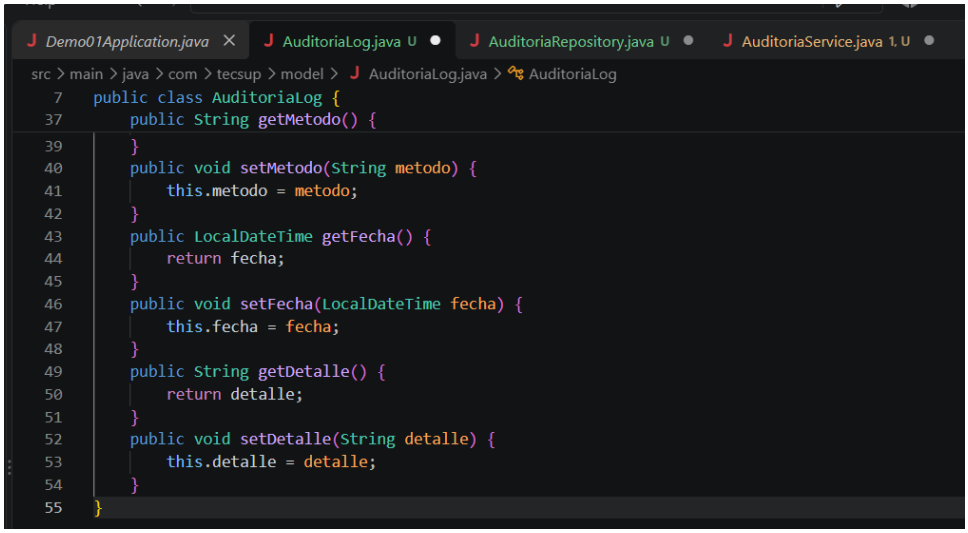


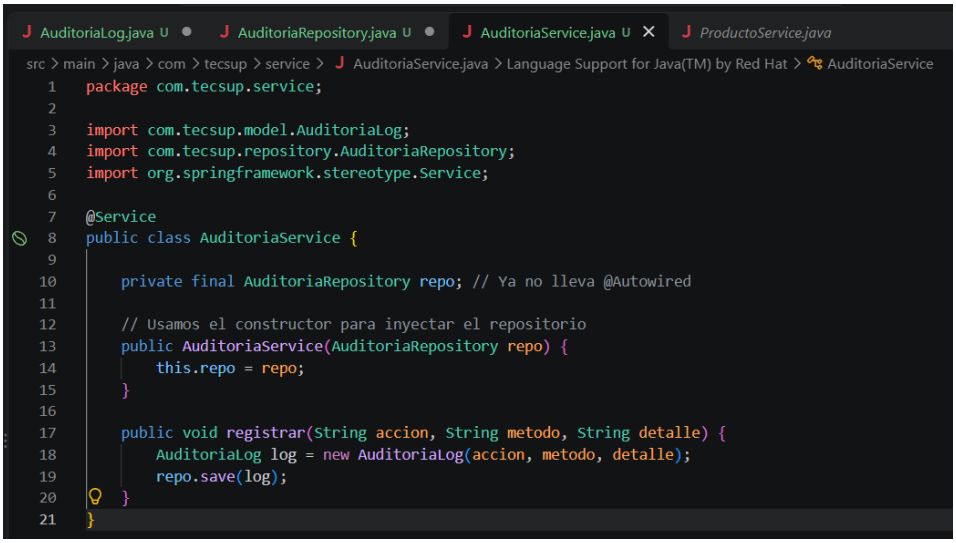
PARTE 1: Auditoría completa

Implementar los siguientes puntos:

- **Para actualizar:**
  - Acción: "ACTUALIZAR"
  - Detalle: incluir el ID del producto actualizado
- **Para listar:**
  - Acción: "LISTAR"
  - Detalle: indicar cantidad de registros obtenidos







PARTE 2: Mejora del AOP (nivel intermedio)

Modificar el aspecto para:

- Capturar parámetros del método (JoinPoint)
- Registrar información dinámica (no texto fijo)

Ejemplo esperado:

"Se actualizó producto con ID: 5"

```
// --- PARTE 2: ACTUALIZAR (Mensaje Dinámico con JoinPoint) ---
@AfterReturning("execution(* com.tecsup.controller.ProductoController.actualizar(..))")
public void auditarActualizar(JoinPoint joinPoint) {
    // Capturamos los parámetros del método. El ID suele ser el primer parámetro (posición 0)
    Object[] args = joinPoint.getArgs();
    Long id = (args.length > 0) ? (Long) args[0] : null;

    auditoriaService.registrar(
            accion: "ACTUALIZAR",
            joinPoint.getSignature().getName(),
            "Se actualizó producto con ID: " + id // Cumple con el ejemplo esperado
    );
}
```

```
// --- PARTE 2: ELIMINAR (Mensaje Dinámico con JoinPoint) ---
@AfterReturning("execution(* com.tecsup.service.ProductoService.eliminar(..))")
public void auditarEliminar(JoinPoint joinPoint) {
    // Capturamos el ID del producto eliminado
    Object[] args = joinPoint.getArgs();
    Long id = (args.length > 0) ? (Long) args[0] : null;

    auditoriaService.registrar(
            accion: "ELIMINAR",
            joinPoint.getSignature().getName(),
            "Se eliminó producto con ID: " + id
    );
}
```

```
// --- PARTE 1 y 2: LISTAR (Mensaje Dinámico capturando el resultado) ---
@AfterReturning(pointcut = "execution(* com.tecsup.service.ProductoService.listar(..))", returning = "lista")
public void auditarListar(JoinPoint joinPoint, Object lista) {
    int cantidad = 0;
    // Verificamos si lo que devolvió el método es una lista para contar su tamaño
    if (lista instanceof List) {
        cantidad = ((List<?>) lista).size();
    }

    auditoriaService.registrar(
            accion: "LISTAR",
            joinPoint.getSignature().getName(),
            "Se obtuvieron " + cantidad + " registros"
    );
}
```

PARTE 3: Manejo de errores con AOP

Modificar ErrorAspect para:

- Registrar errores en la base de datos (tabla auditoría)
- No solo mostrar en consola

ErrorAspect.java U

src > main > java > com > tecsup > aspect > ErrorAspect.java > Language Support for Java(TM) by Red Hat > ErrorAspect

```java
package com.tecsup.aspect;

import com.tecsup.service.AuditoriaService;
import org.aspectj.lang.annotation.*;
import org.springframework.stereotype.Component;

@Aspect
@Component
public class ErrorAspect {

    private final AuditoriaService auditoriaService;

    // Inyección de dependencias por constructor
    public ErrorAspect(AuditoriaService auditoriaService) {
        this.auditoriaService = auditoriaService;
    }

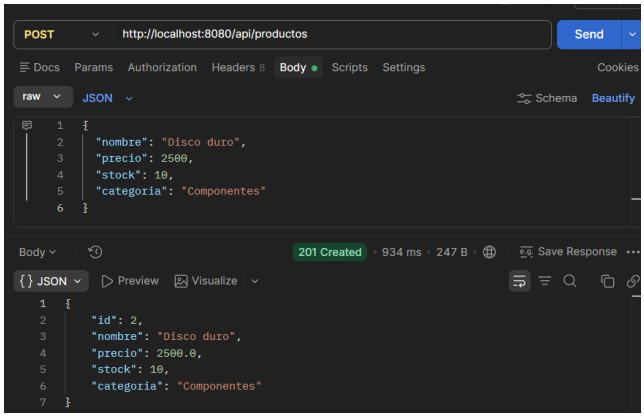
    @AfterThrowing(
            pointcut = "execution(* com.tecsup.service.*.*(..))",
            throwing = "ex"
    )
    public void capturarError(Exception ex) {
        // 1. Mantiene el comportamiento original de mostrar el error en consola
        System.out.println("ERROR AOP: " + ex.getMessage());

        // 2. PARTE 3: Registra dinámicamente el error en la base de datos
        auditoriaService.registrar(
                accion: "ERROR",
                metodo: "Fallo en el sistema",
                ex.getMessage() // Captura el mensaje exacto de la excepción
        );
    }
}
```

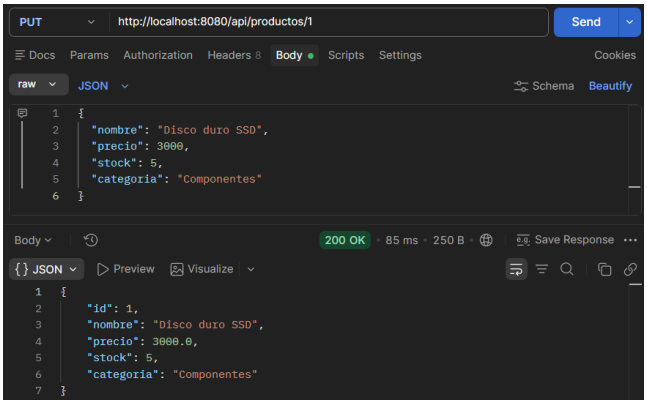
PARTE 4: Validación

Probar en POSTMAN:

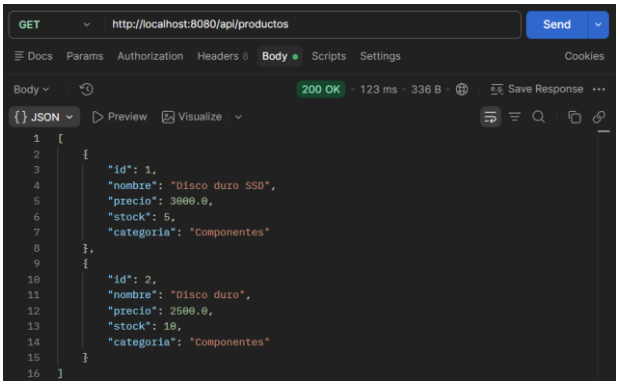
- POST (crear)



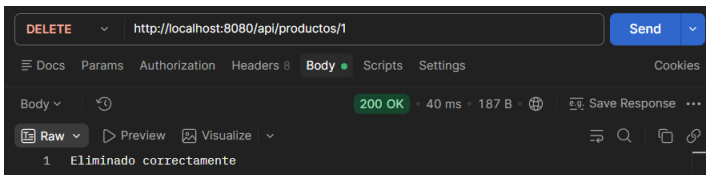
- PUT (actualizar)



- GET (listar)



- DELETE (eliminar)



Luego:

Verificar que todos los eventos se registren en la tabla **auditoria_log**



Mostrando filas 0 - 4 (total de 5, La consulta tardó 0,0002 segundos.)

```
SELECT * FROM `auditoria_log`
```

Perfilando [Editar en línea] [Editar] [Explicar SQL] [Crear código PHP] [Actualizar]

Mostrar todo | Número de filas: 25 Filtrar filas: Buscar en esta tabla Ordenar según la clave: Ninguna

Opciones extra

	id	accion	metodo	fecha	detalle
Editar Copiar Borrar	1	CREAR	guardar	2026-09-28 02:44:32	Se registró un producto
Editar Copiar Borrar	2	CREAR	guardar	2026-09-28 02:45:27	Se registró un producto
Editar Copiar Borrar	3	ACTUALIZAR	actualizar	2026-09-28 02:45:27	Se actualizó producto con ID: 1
Editar Copiar Borrar	4	LISTAR	listar	2026-09-28 02:46:08	Se obtuvieron 2 registros
Editar Copiar Borrar	5	ELIMINAR	eliminar	2026-09-28 02:46:39	Se eliminó producto con ID: 1

OBSERVACIONES: *(Las observaciones son las notas aclaratorias y problemas que se pudo presentar en el desarrollo del laboratorio)*

Mencionar 5 observaciones:

- **Me di cuenta de que usar joinPoint.getArgs() es súper útil porque me permitió jalar el ID directamente desde la petición de Postman sin tener que escribir el texto a mano en el código.**
- **Al configurar el ErrorAspect, vi que los errores ya no se pierden en la consola negra del editor, sino que quedan guardados en la tabla de la base de datos, lo cual ayuda mucho a rastrearlos.**
- **Tuve que tener cuidado al auditar la actualización. Si lo ponía en el servicio, chocaba con el método guardar y me registraba una creación falsa, así que lo apunté directo al controlador y funcionó perfecto.**
- **Mi editor me tiraba una línea amarilla molestosa por usar @Autowired en los servicios. Lo solucioné rápido cambiando el código para inyectar por constructor y se vio mucho más limpio.**
- **Hacer las pruebas en Postman fue bastante directo; mientras yo mandaba mis peticiones normales (POST, GET...), me pareció genial ver cómo la base de datos registraba todo por detrás sin que yo hiciera nada extra en la API.**

CONCLUSIONES: *(Las conclusiones son una opinión sobre tu trabajo, explicar cómo resolviste las dudas o problemas presentados en el laboratorio. Además de aportar una opinión critica de lo realizado)*

Indicar 5 conclusiones:

- **Entendí que la Programación Orientada a Aspectos es una maravilla. Pude meterle todo un sistema de auditoría a mi proyecto sin tocar ni ensuciar el código que ya tenía funcionando en los controladores.**
- **Lograr que los mensajes sean dinámicos (como *"Se actualizó producto con ID: 1"*) hace que este laboratorio pase de ser un simple registro a una herramienta de monitoreo que sí usaría en un trabajo real.**
- **Guardar físicamente el ex.getMessage() de los errores en MySQL me enseñó que tener un historial de fallos te salva la vida cuando toca darle mantenimiento a un sistema.**
- **Quitar los @Autowired y usar constructores no solo me apagó las alertas del IDE, sino que me ayudó a entender cómo hacer un código más moderno y apegado a las buenas prácticas de Spring Boot.**
- **Comprobé que probar el CRUD completo en Postman me dio la seguridad total de que mis "aspectos" estaban interceptando cada petición en el momento exacto, demostrando que toda la arquitectura funciona.**